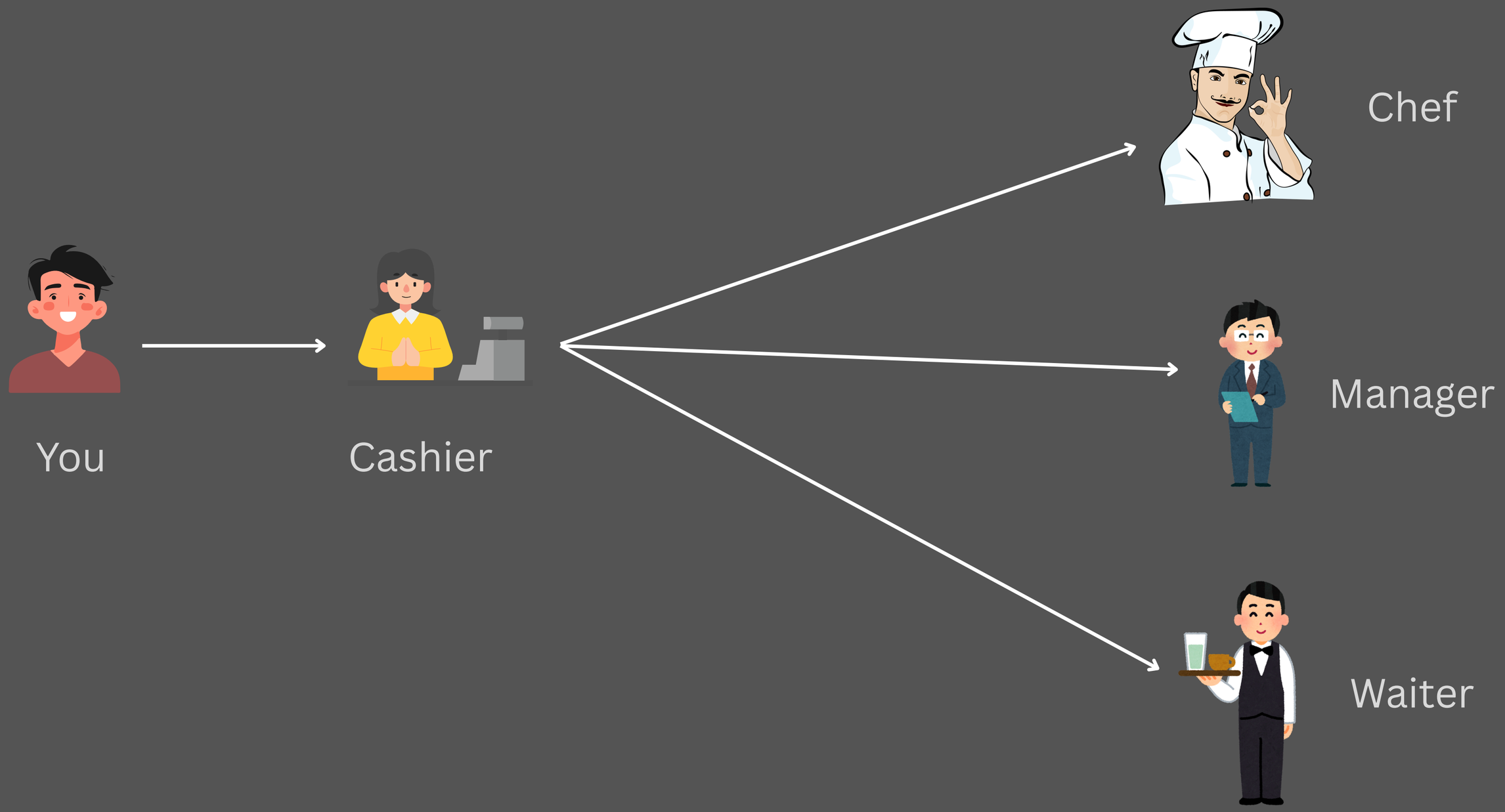
APACHE KAFKA MASTERCLASS

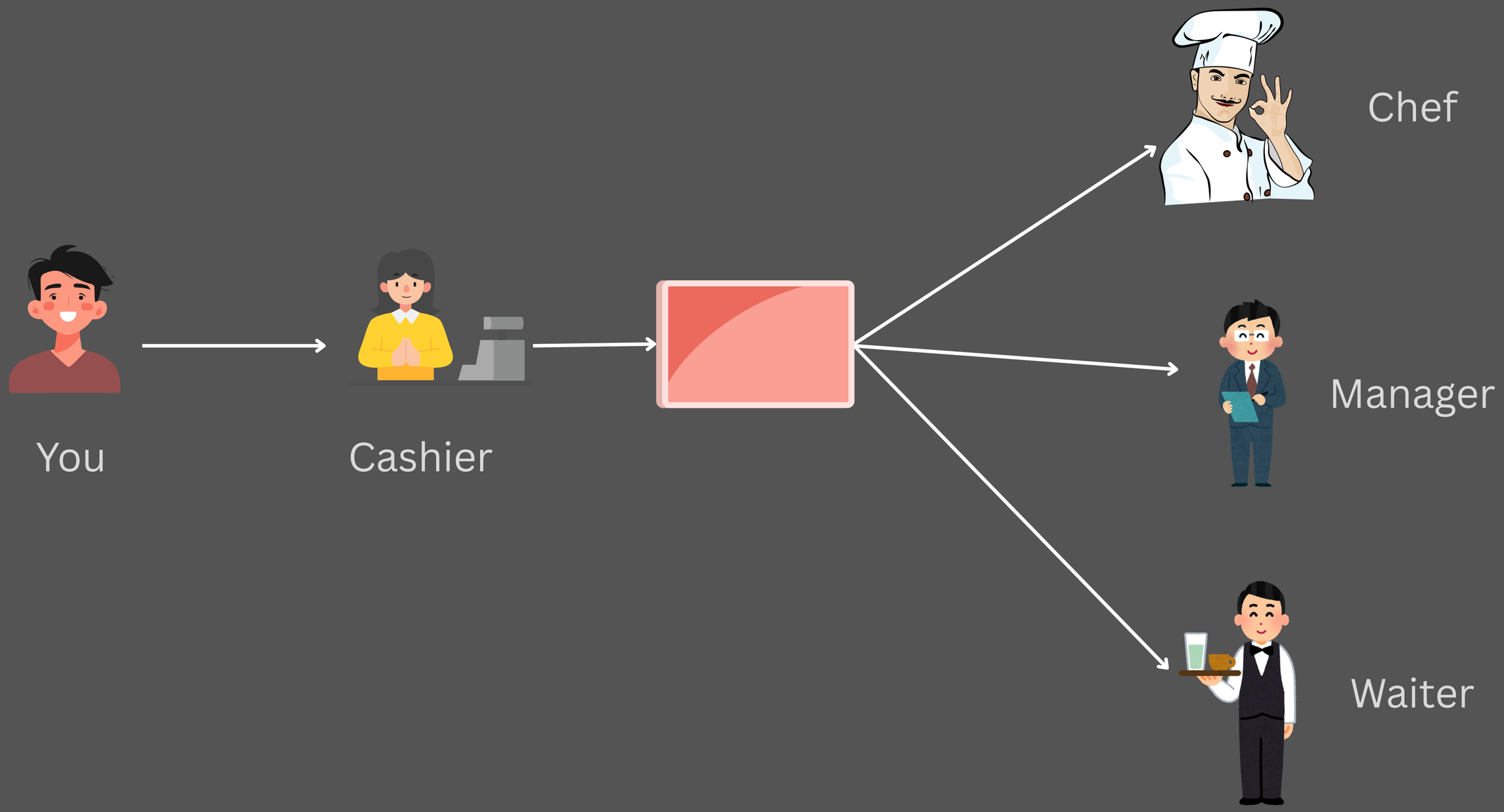
Apache Kafka is an open-source distributed event streaming platform

Apache Kafka is an open-source distributed event streaming platform

EmbarkX.com



EmbarkX.com



Kafka helps by

- Decoupling services
- Scalability
- Replayability
- Efficiency

Event streaming means continuously capturing events (data) in real-time, storing them in a log (like a timeline), and allowing systems to react to them — either immediately or later.

PRODUCER
KAFKA
CONSUMER
CONSUMER

Method	Description	Example
Request/Response	One system calls another and waits for a reply	REST API, HTTP
Batch Processing	System processes large chunks of data at intervals	Nightly data imports, reports
Message Queues	One system sends a message to a queue, another pulls from it	RabbitMQ, SQS
Event Streaming	Systems send and react to a constant flow of events	Kafka, Pulsar

Kafka is like a **WhatsApp** group

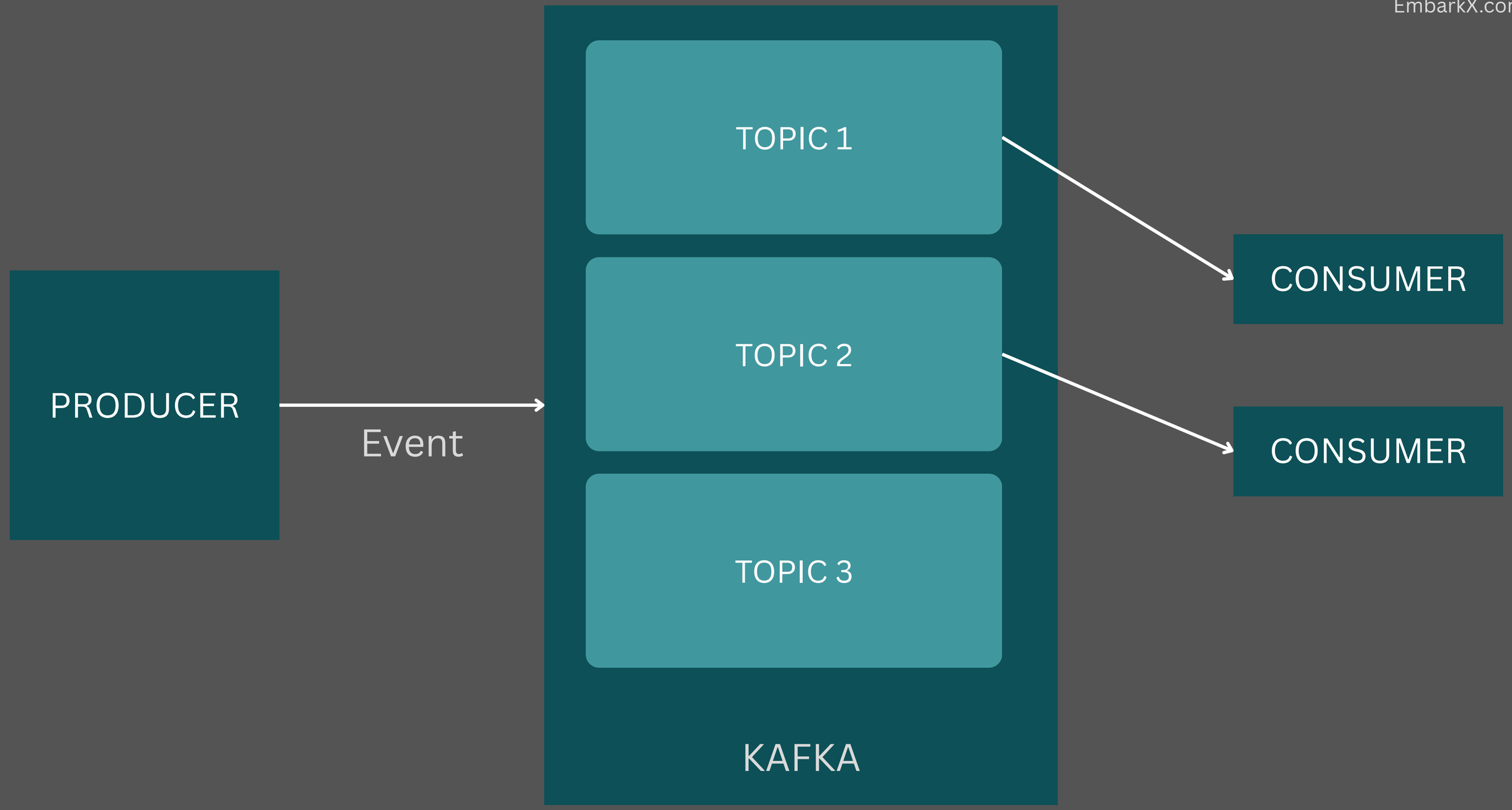
Feature	Kafka	RabbitMQ
Model	**Event streaming** (log-based)	**Message queuing**
Message storage	Keeps all messages for a set time (you can reread)	Deletes message after it's consumed
Consumers	Each consumer group can read **independently**	Messages are **removed** once delivered
Best for	High-throughput, real-time analytics, data replay	Task queueing, short-lived jobs
Persistence	Designed for **long-term storage and replay**	Focused on **short-term delivery**
Ordering	Guarantees order within a partition	No strict ordering unless manually handled
Throughput	Very high (designed for streaming big data)	Moderate

Kafka Architecture and Components

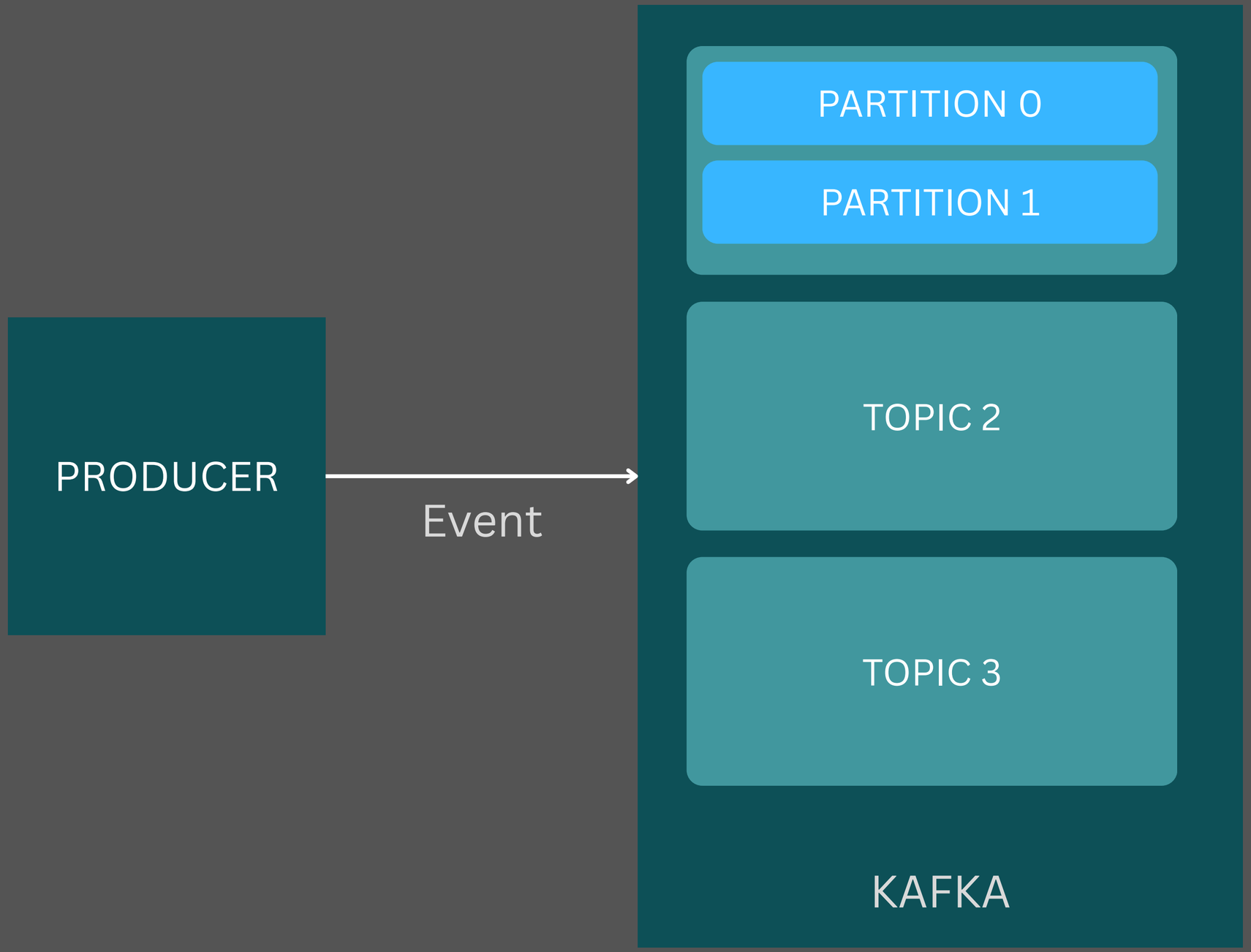
- Event
- Producer
- Consumer
- Topics
- Partitions
- Consumer Groups
- Offset
- Consumer Rebalancing

PRODUCER
Event
KAFKA
CONSUMER
CONSUMER

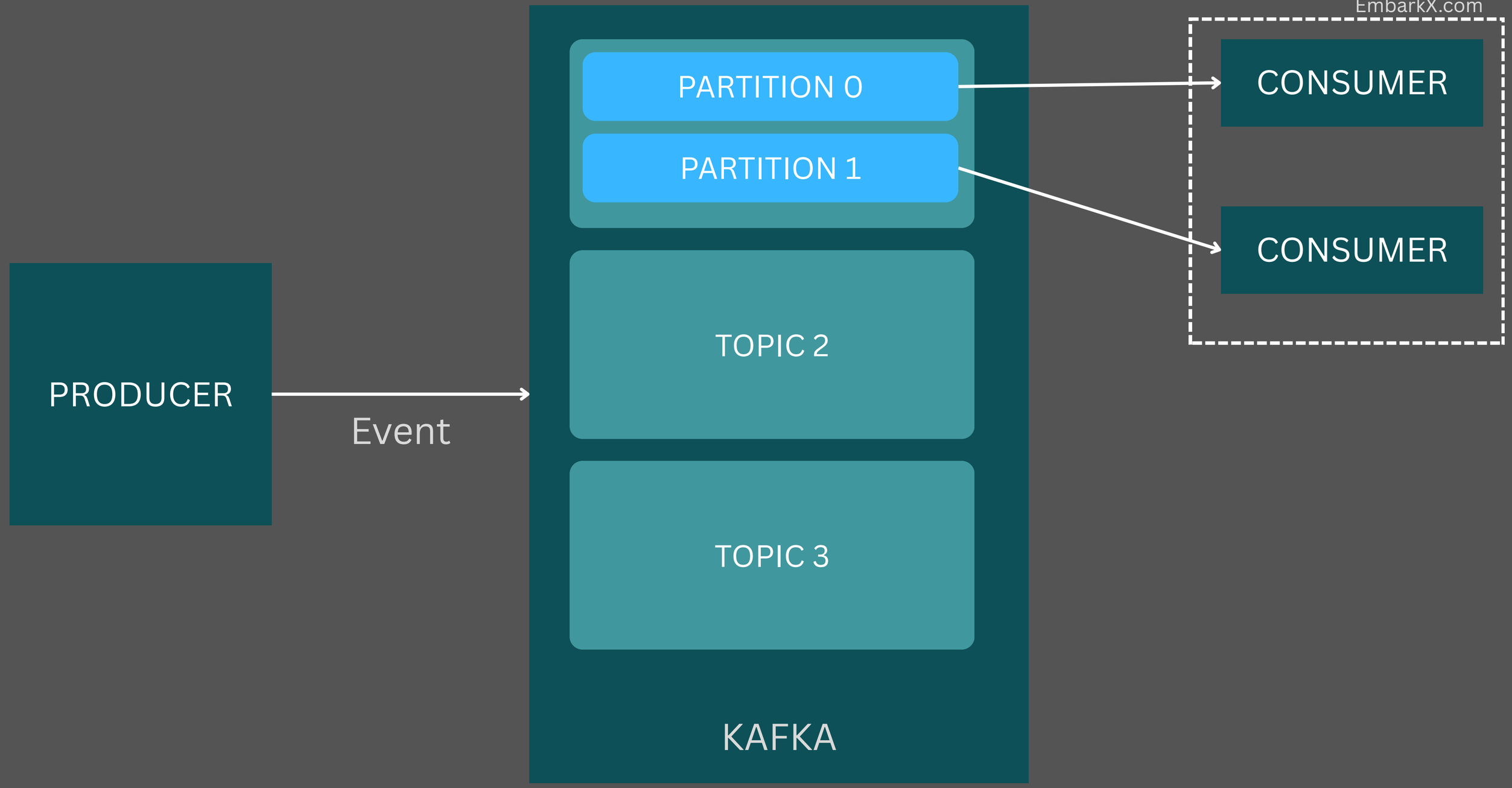
EmbarkX.com


PRODUCER
Event
TOPIC 1
TOPIC 2
TOPIC 3
KAFKA
CONSUMER
CONSUMER

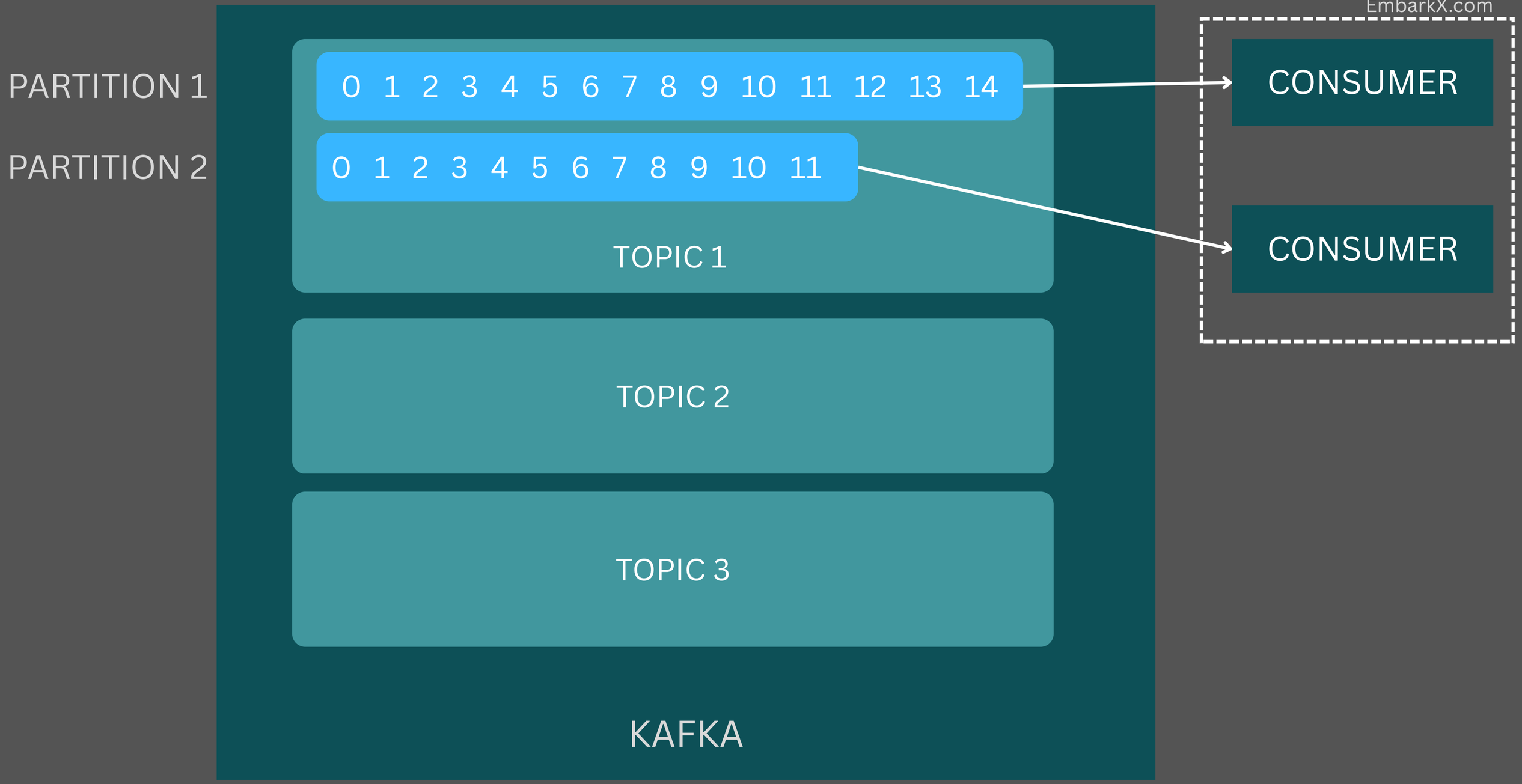
EmbarkX.com

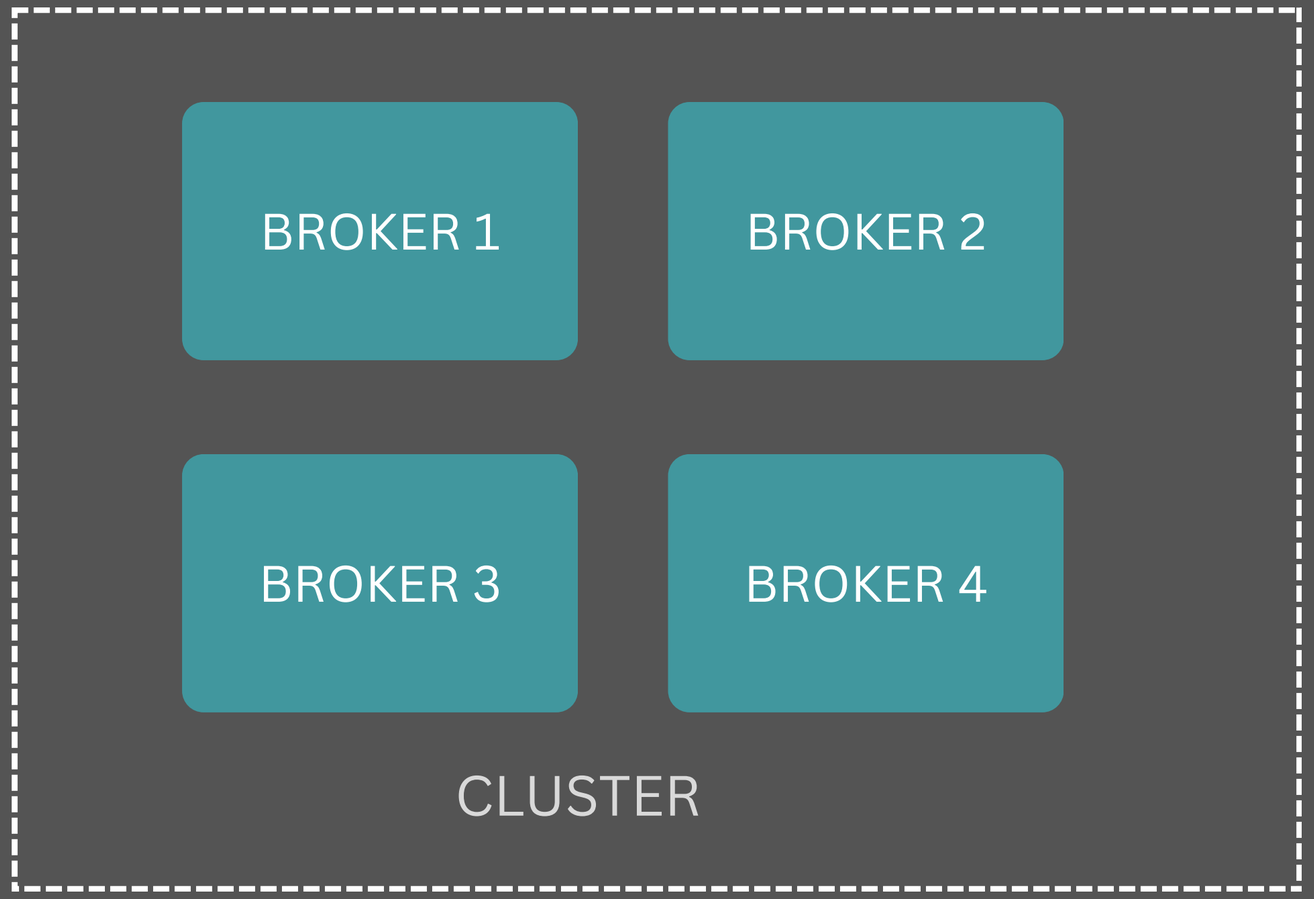

PARTITION 0
PARTITION 1
TOPIC 2
TOPIC 3
KAFKA
PRODUCER
Event

EmbarkX.com

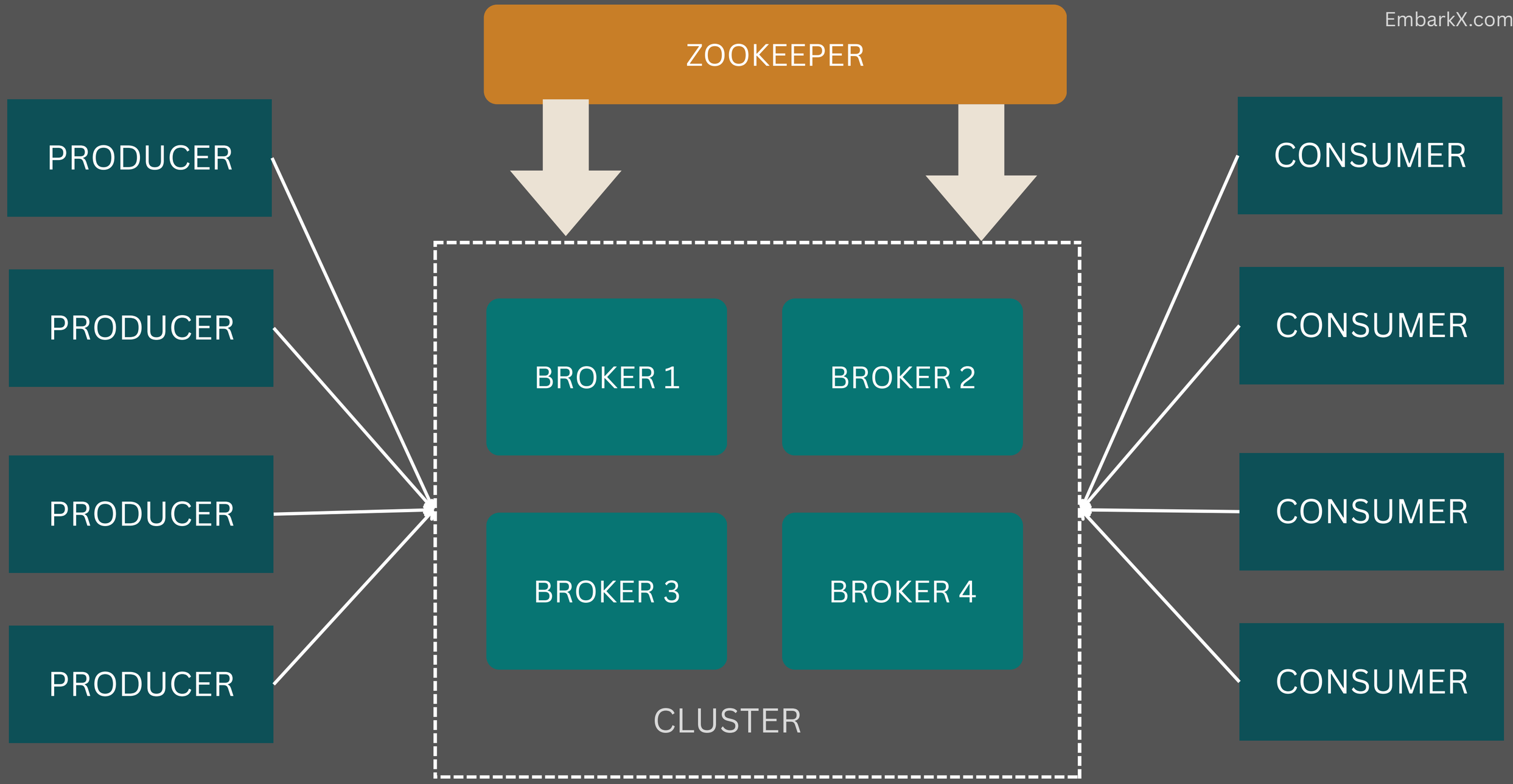

PRODUCER
Event
PARTITION 0
PARTITION 1
TOPIC 2
TOPIC 3
KAFKA
CONSUMER
CONSUMER

EmbarkX.com


PARTITION 1
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
PARTITION 2
0 1 2 3 4 5 6 7 8 9 10 11
TOPIC 1
TOPIC 2
TOPIC 3
KAFKA
CONSUMER
CONSUMER


BROKER 1
BROKER 2
BROKER 3
BROKER 4
CLUSTER

EmbarkX.com


ZOOKEEPER
PRODUCER
PRODUCER
PRODUCER
PRODUCER
BROKER 1
BROKER 2
BROKER 3
BROKER 4
CLUSTER
CONSUMER
CONSUMER
CONSUMER
CONSUMER

Term	Meaning in WhatsApp Analogy	Definition
Event	A message sent	A single unit of data or message that flows through Kafka.
Producer	The sender of the message	An application or service that publishes events to Kafka topics.
Consumer	The one who reads the message	An application or service that subscribes to Kafka topics to read messages.
Topic	A group chat (e.g., “Orders”, “Payments”)	A category or feed name to which records are sent by producers.
Partition	Sub-channels in a group for faster delivery	A topic can be split into partitions to enable parallel processing and scaling.

Term	Meaning in WhatsApp Analogy	Definition
Consumer Group	Team of people processing messages together	A group of consumers sharing the workload of reading from a topic.
Offset	Bookmark of last message read	A position or marker in a partition that tracks how much a consumer has read.
Broker	WhatsApp server storing and routing messages	A Kafka server that stores and serves Kafka messages.
Zookeeper	Admin who keeps track of who’s doing what	Manages metadata, leader election, and configuration in Kafka (soon deprecated).

Concept	Beginner Summary
Retention	How long Kafka stores messages (like chat history)
Replication	Kafka keeps copies of messages for safety
Producer Acknowledgement	Ways to confirm if message was successfully sent
Dead Letter Queue (DLQ)	Special place to store failed messages
Kafka Connect	Tool to connect Kafka with databases, cloud, etc.
Kafka Streams	Allows you to process data inside Kafka in real-time

Apache Kafka is an open-source distributed event streaming platform

@FunctionalInterface

A Functional Interface in Java is an interface that contains exactly one abstract method.

```
@FunctionalInterface
public interface MyInterface {
    void doSomething(); // only one method
}
```

Built-In Functional Interfaces

- Function<T, R>
- Consumer<T>
- Supplier<T>
- Predicate<T>

Function<T, R>

Takes an input of type T, returns something of type R

```
Function<String, Integer> getLength = s -> s.length();

System.out.println(getLength.apply("hello")); // prints 5
```

Consumer<T>

Takes input T, does something, but returns nothing.

```
Consumer<String> printer = s -> System.out.println("Message: " + s);

printer.accept("hello world"); // prints Message: hello world
```

Supplier<T>

Takes no input, returns a value of type T

```
Supplier<String> giveMessage = () -> "Hello from supplier!";

System.out.println(giveMessage.get()); // prints Hello from supplier!
```

Predicate<T>

Takes input T, returns true or false

```
Predicate<String> isLong = s -> s.length() > 5;

System.out.println(isLong.test("hello"));     // false
System.out.println(isLong.test("helloooo"));  // true
```

Spring Cloud Function

Lets you write just one Java function—a Supplier, Function or Consumer—then bind it to HTTP, Kafka, or Lambda by configuration alone.

Spring Cloud Stream

Lets you write plain Java functions and then “bind” them to messaging destinations (topics, queues) without writing any broker-specific code.

Spring Boot Application
Spring Cloud Stream
RabbitMQ
Kafka